

CSC 491/591

Self-Driving Cars: Theory and Practice

Machine Learning and Object Detection (Lab)

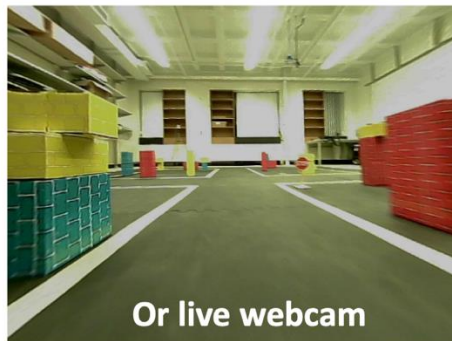
Dhruva Ungrupulithaya

Department of Computer Science

NC State University

Why do we need Machine Learning?

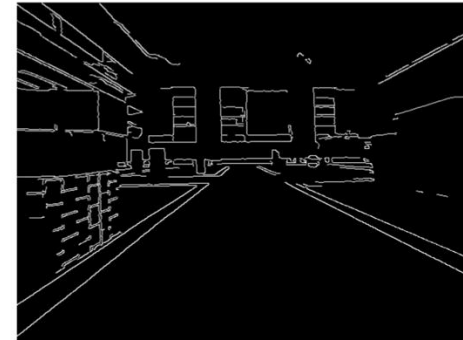
- Canny edge detection and Hough transform



Grayscale

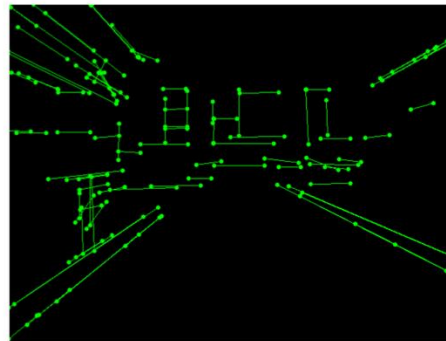


Canny edge detection

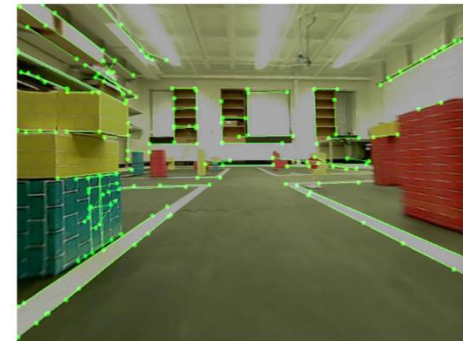


Experiment with various values for
threshold,
minLineLength,
maxLineGap

Use `rho=1, theta=np.pi/180`



Hough transform



Overlay

What is Machine Learning?

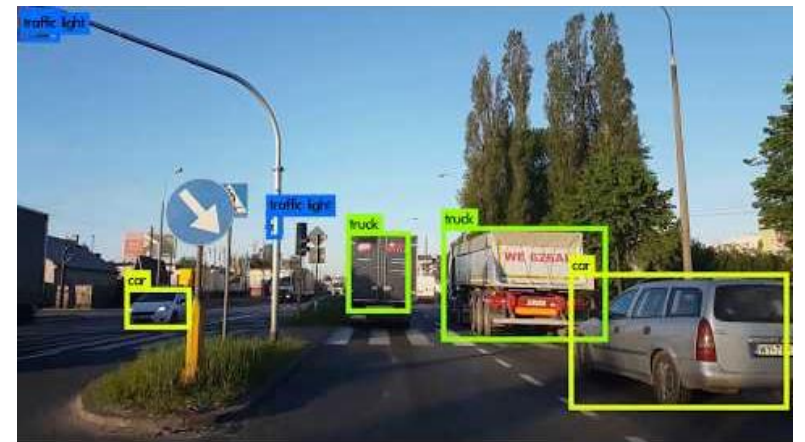
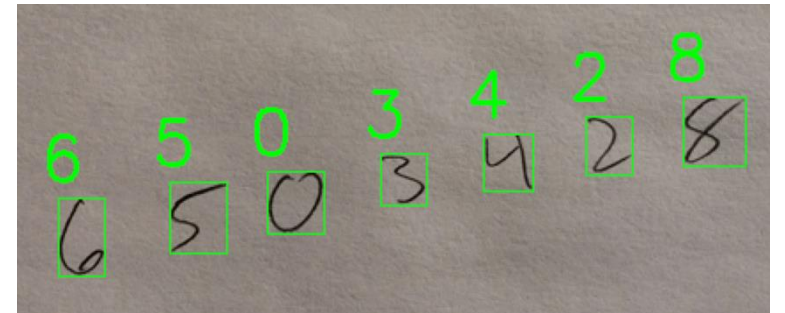
- Learning from data
 - Learn a complex task from large quantities of examples
 - Predict/produce inferences on new data/situation

[Machine Learning is the] field of study that gives computers the ability to learn without being explicitly programmed.

Arthur Samuel, 1959

A computer program is said to learn from experience E with respect to some task T and some performance measure P , if its performance on T , as measured by P , improves with experience E .

Tom Mitchell, 1997



YOLO (You Only Look Once) Real-Time Object Detection

ML in Autonomous Driving



Truth: 22
Prediction: 22



Truth: 52
Prediction: 37



Truth: 39
Prediction: 39



Truth: 22
Prediction: 22



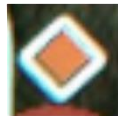
Truth: 32
Prediction: 32



Truth: 18
Prediction: 18



Truth: 32
Prediction: 32



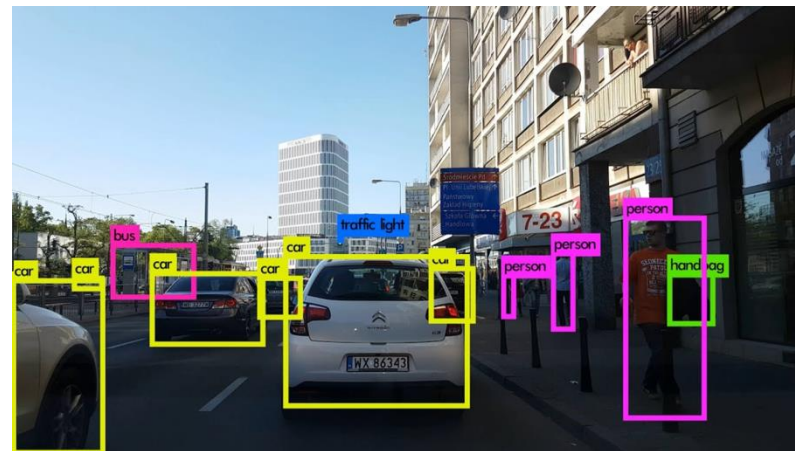
Truth: 61
Prediction: 7



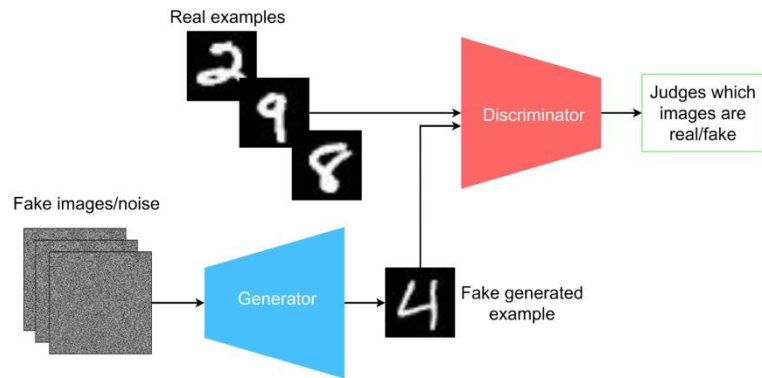
Truth: 17
Prediction: 17



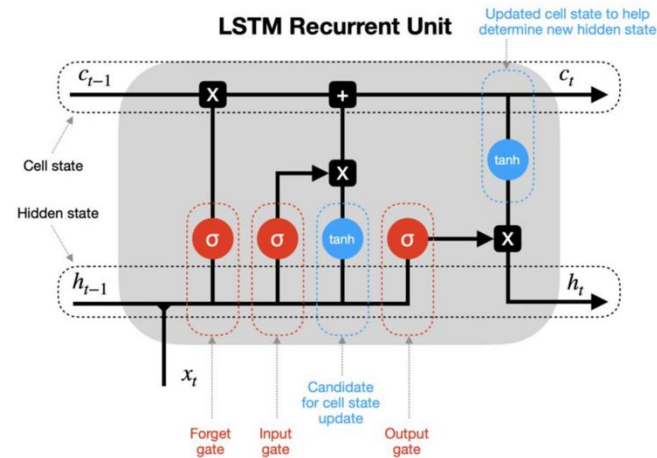
Truth: 53
Prediction: 53



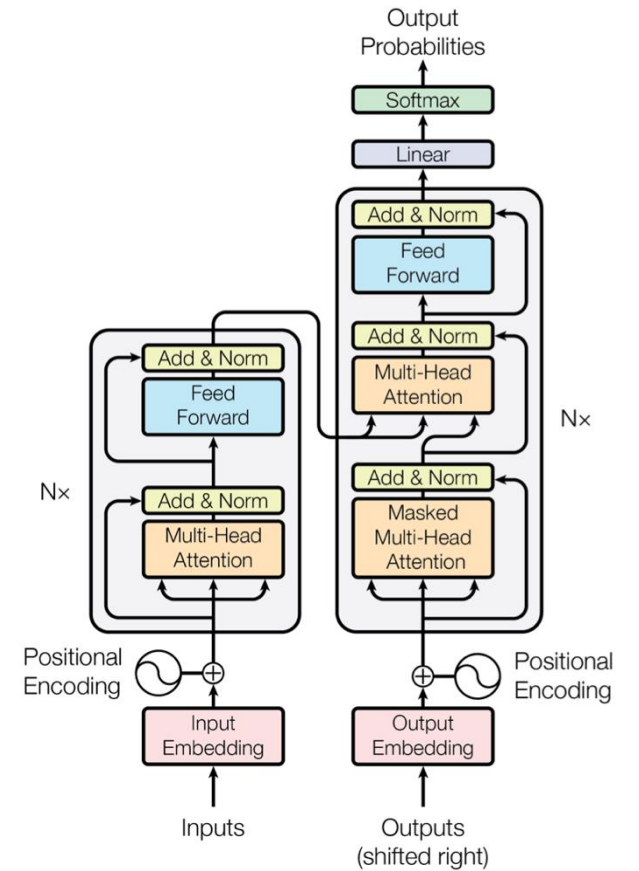
Model Architectures



Generative Adversarial Networks (GANs)



Long-Short Term Memory (LSTM)



Transformers

Convolutional Neural Networks (CNN)

- Most popular architecture for image tasks
- Convolutional (+activation), pooling, convolutional (+activation), pooling, ..., fully connected

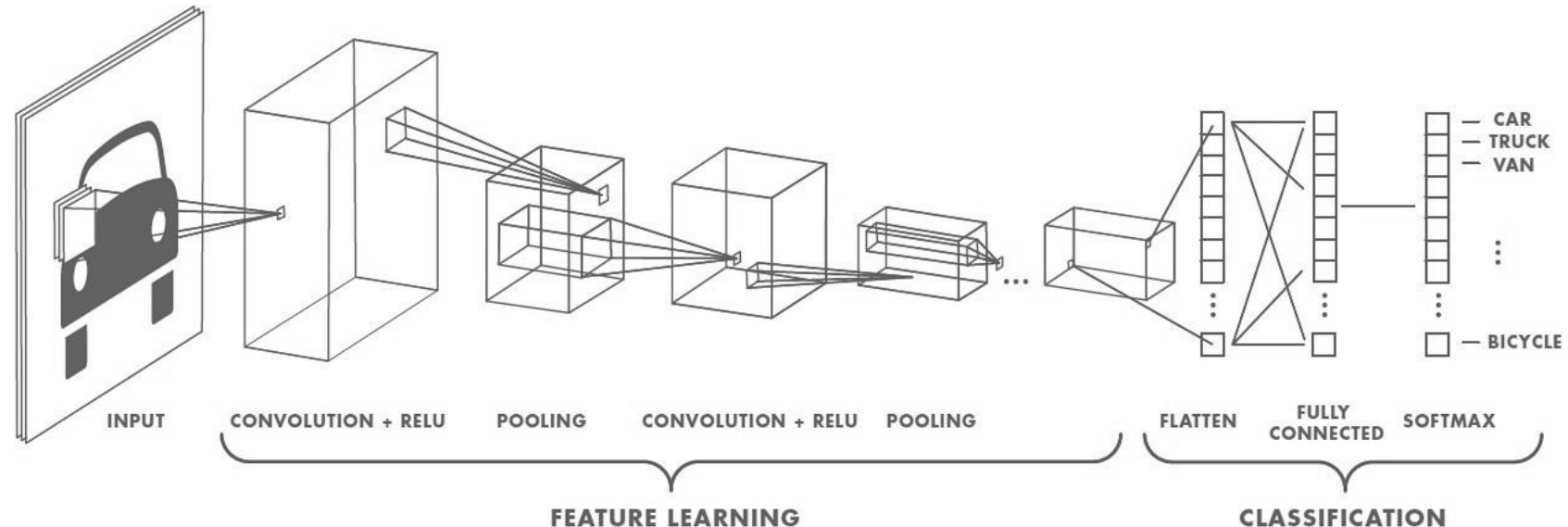
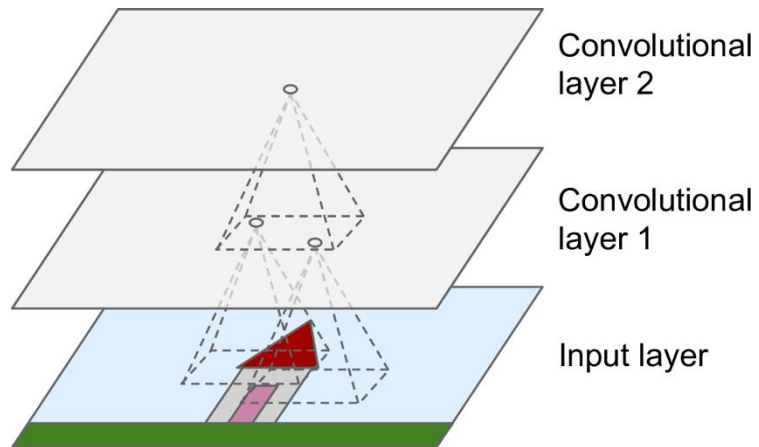


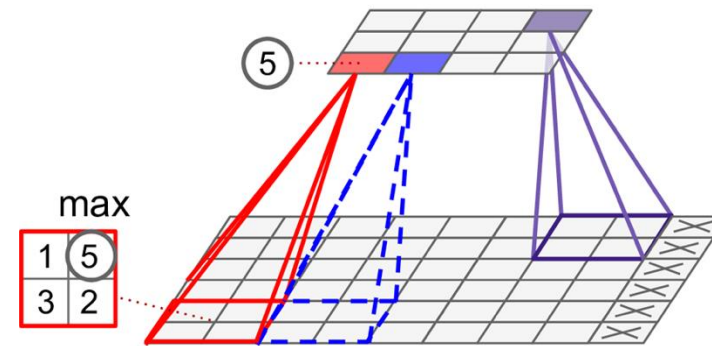
Image source: <https://medium.com/intro-to-artificial-intelligence/semantic-segmentation-udaitys-self-driving-car-engineer-nanodegree-c01eb6eaf9d>

Basic Building Blocks of CNN

Convolutional layer

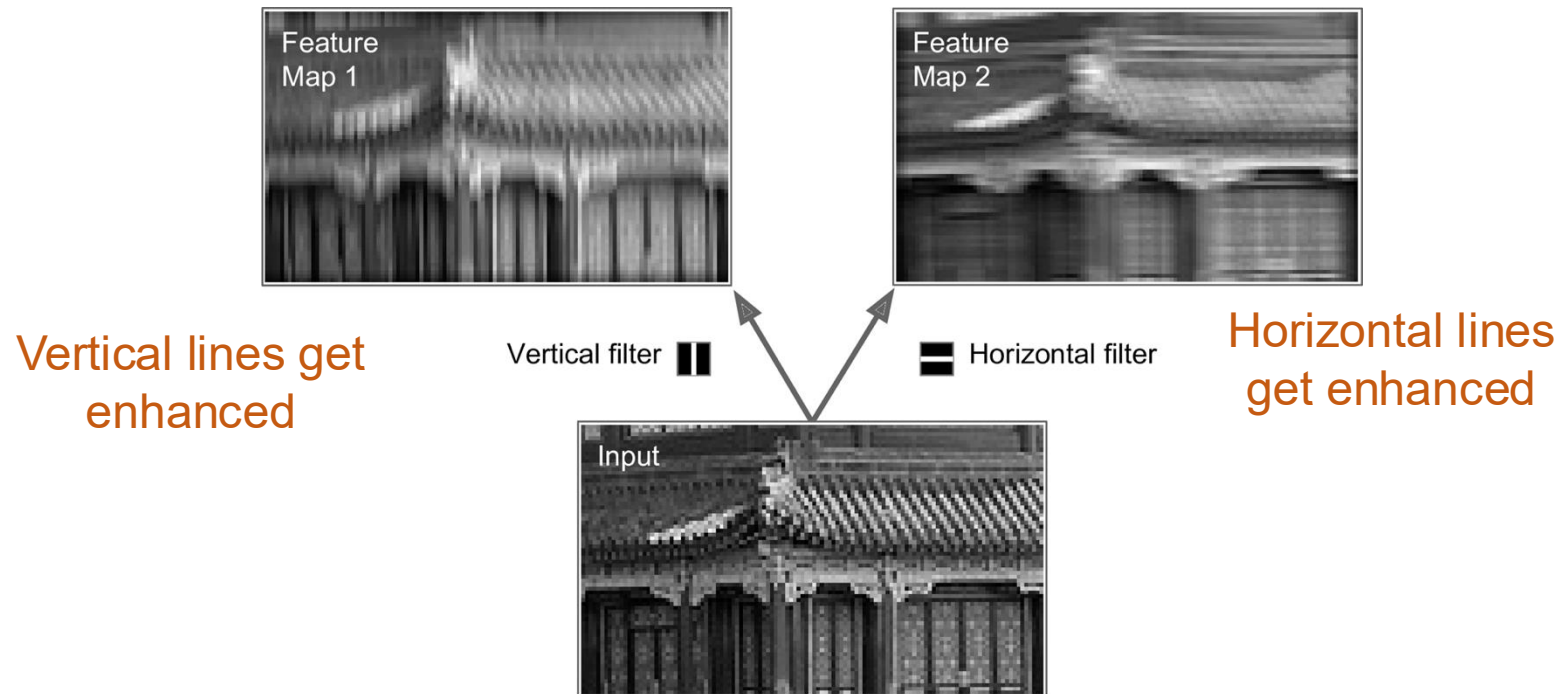


Pooling layer



Convolutional Layer

- Filters (=convolutional kernels)
 - Extracts “features” from the image
 - Complex patterns are found by combining filters
 - CNN training == finding most useful filters (i.e., kernels)



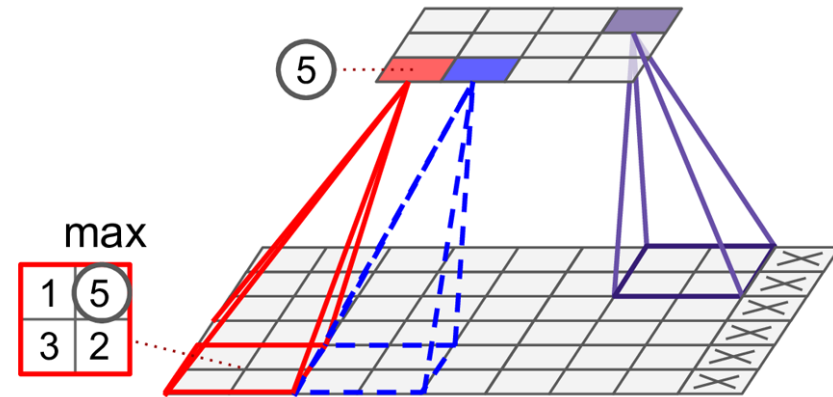
Pooling Layer

- For subsampling (→ reduces dimensionality of feature maps)
 - Aggregates the inputs using an aggregation function such as 'max' or 'mean'
 - Retains the most important information
 - No 'weights'

- Example: Max pooling layer
 - Takes the max input value
(=strongest feature)

2x2 pooling kernel & Stride of 2

→ Drops $\frac{3}{4}$ of inputs



JOSEPH
REDMON

ROSS
GIRSHICK

SANTOSH
DIVVALA

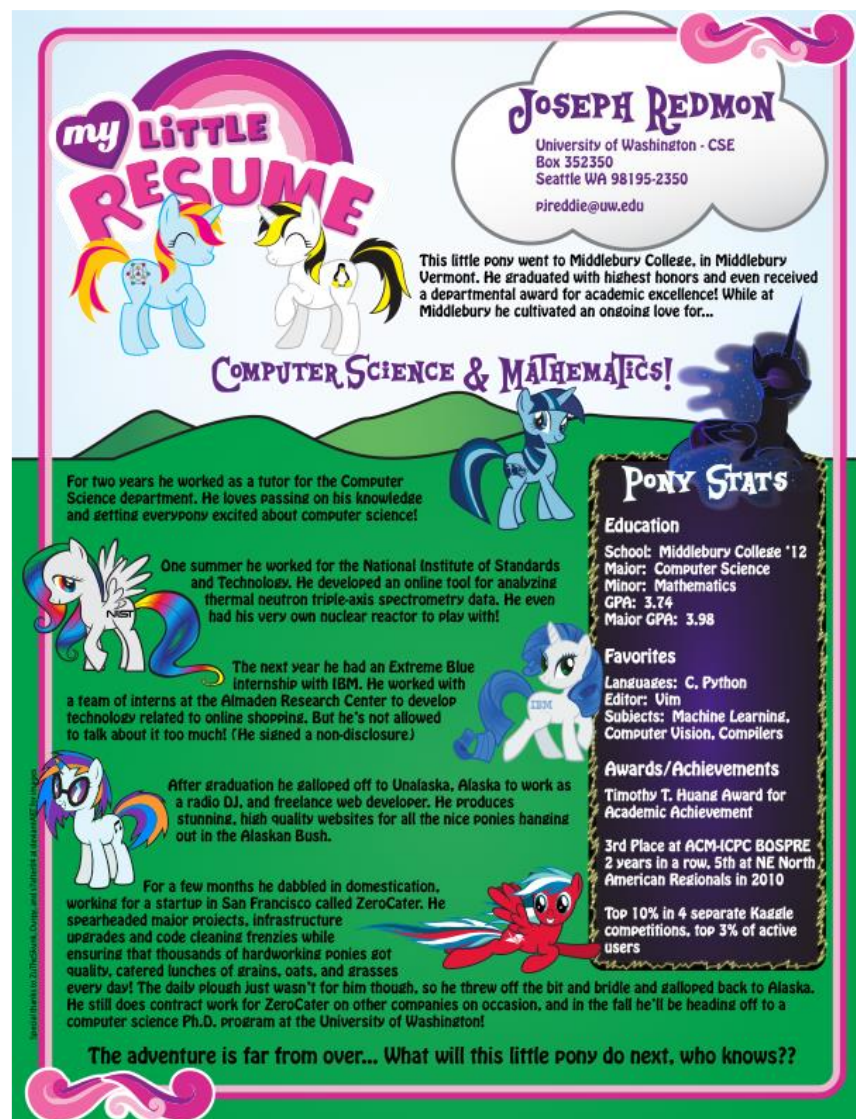
ALI
FARHADI

Dog



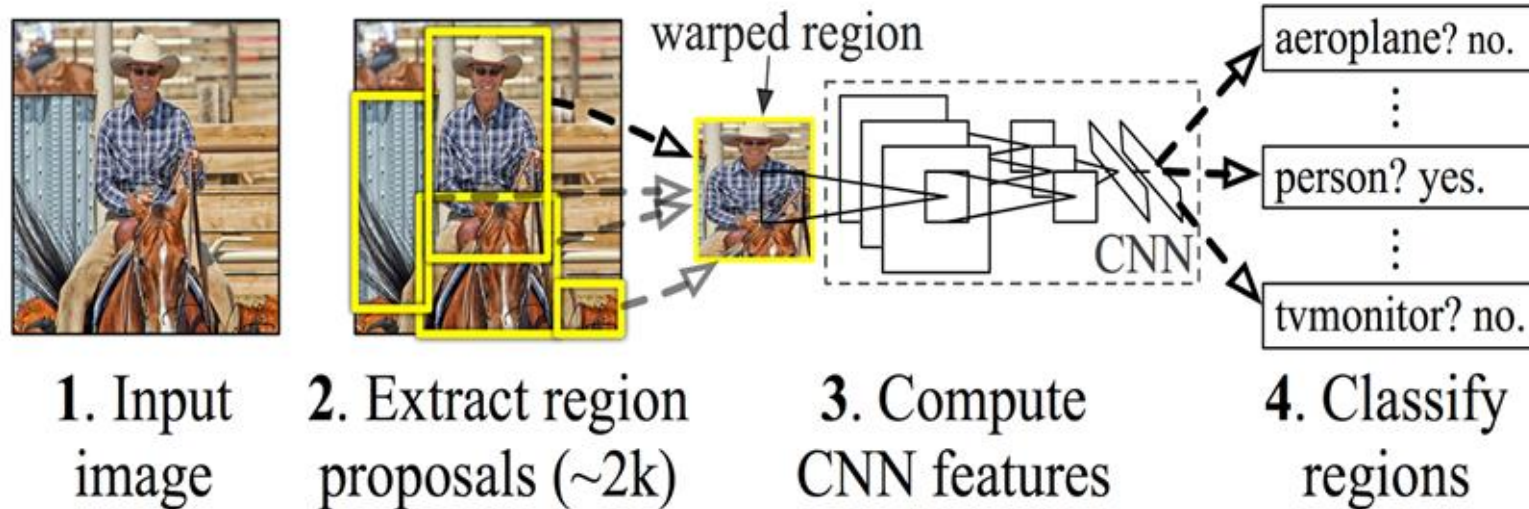
“YOU ONLY LOOK ONCE”
REAL-TIME
DETECTION

Author's Resume...

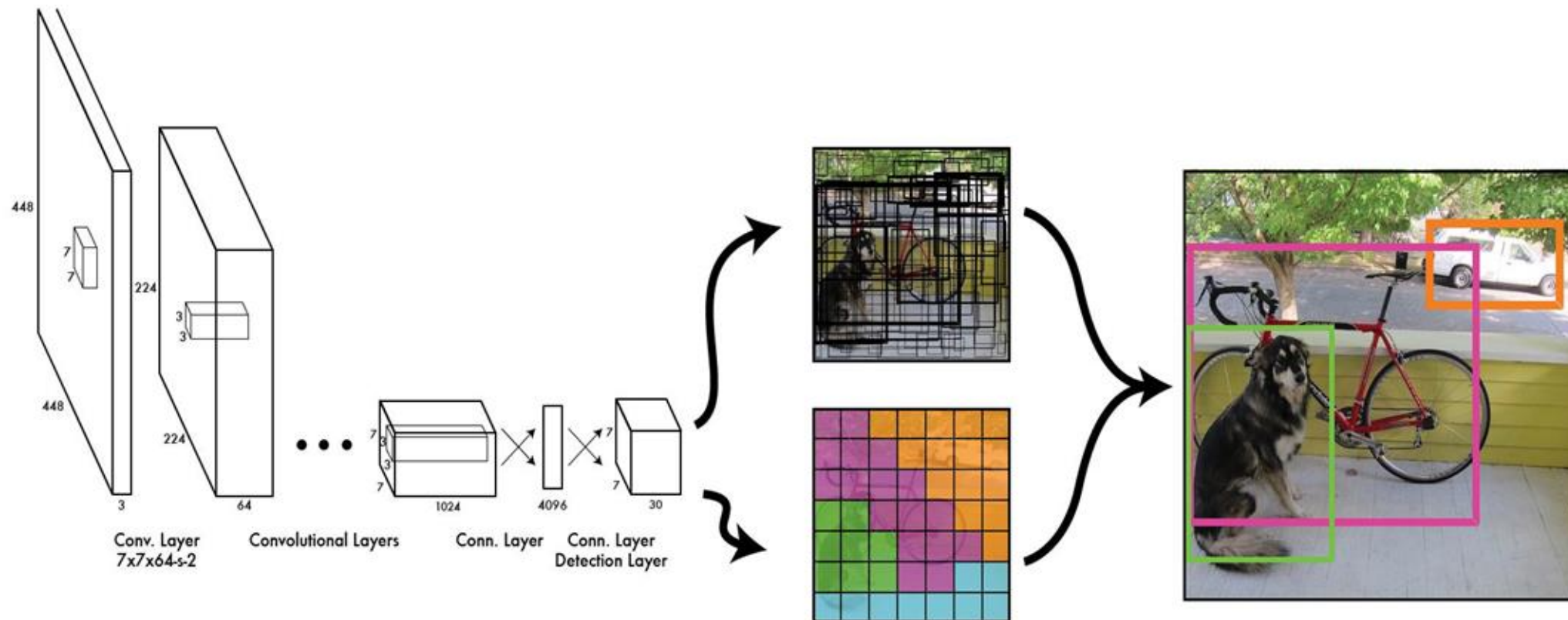


Recurrent CNNs

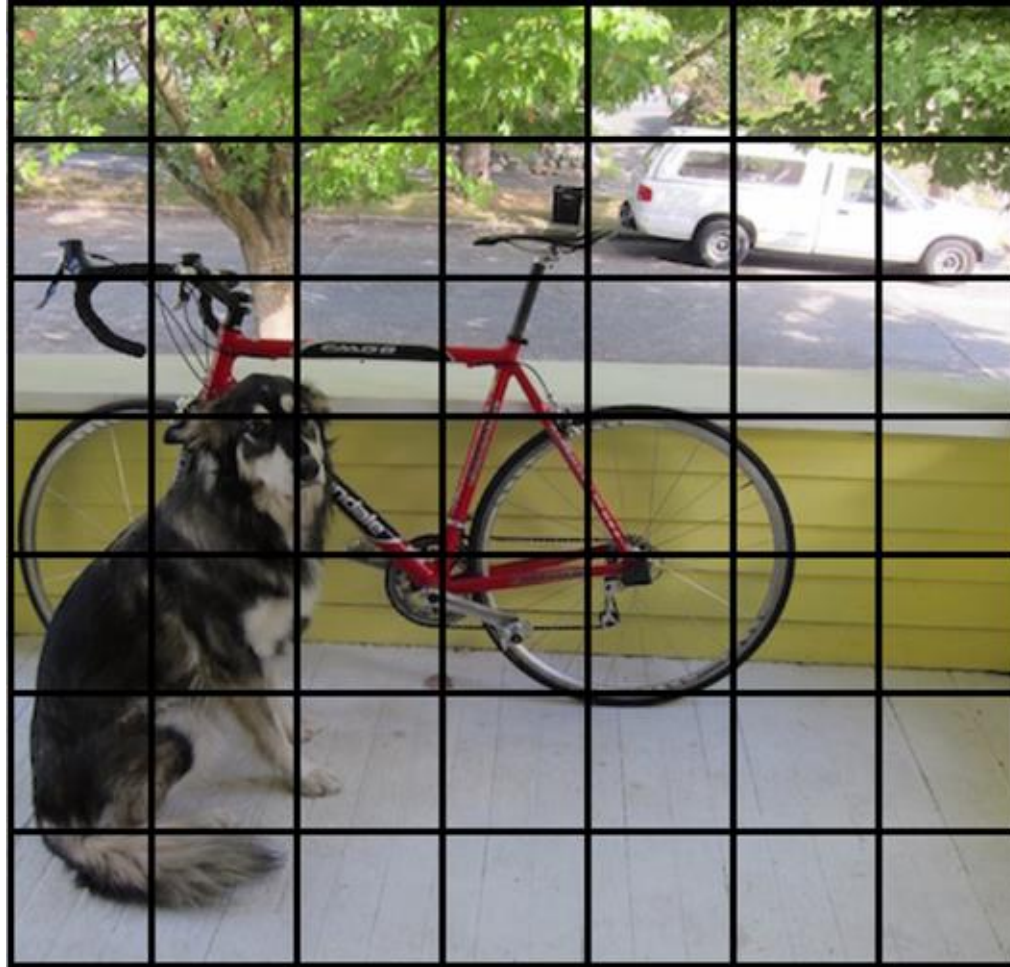
R-CNN: *Regions with CNN features*



You Only Look Once

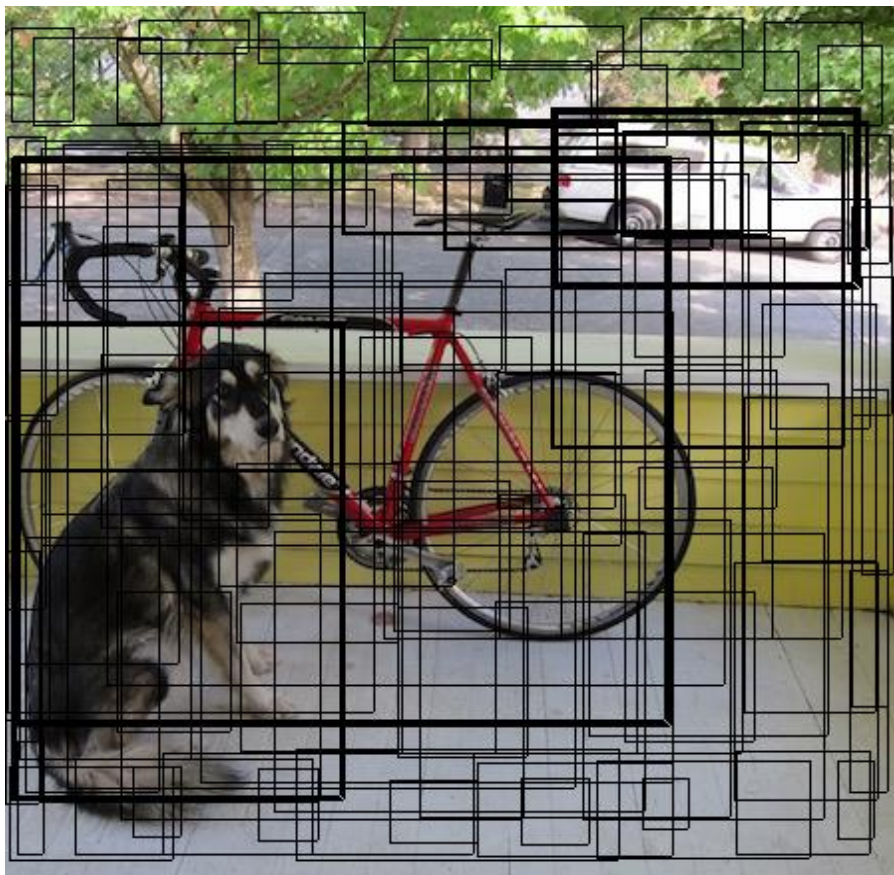


You Only Look Once

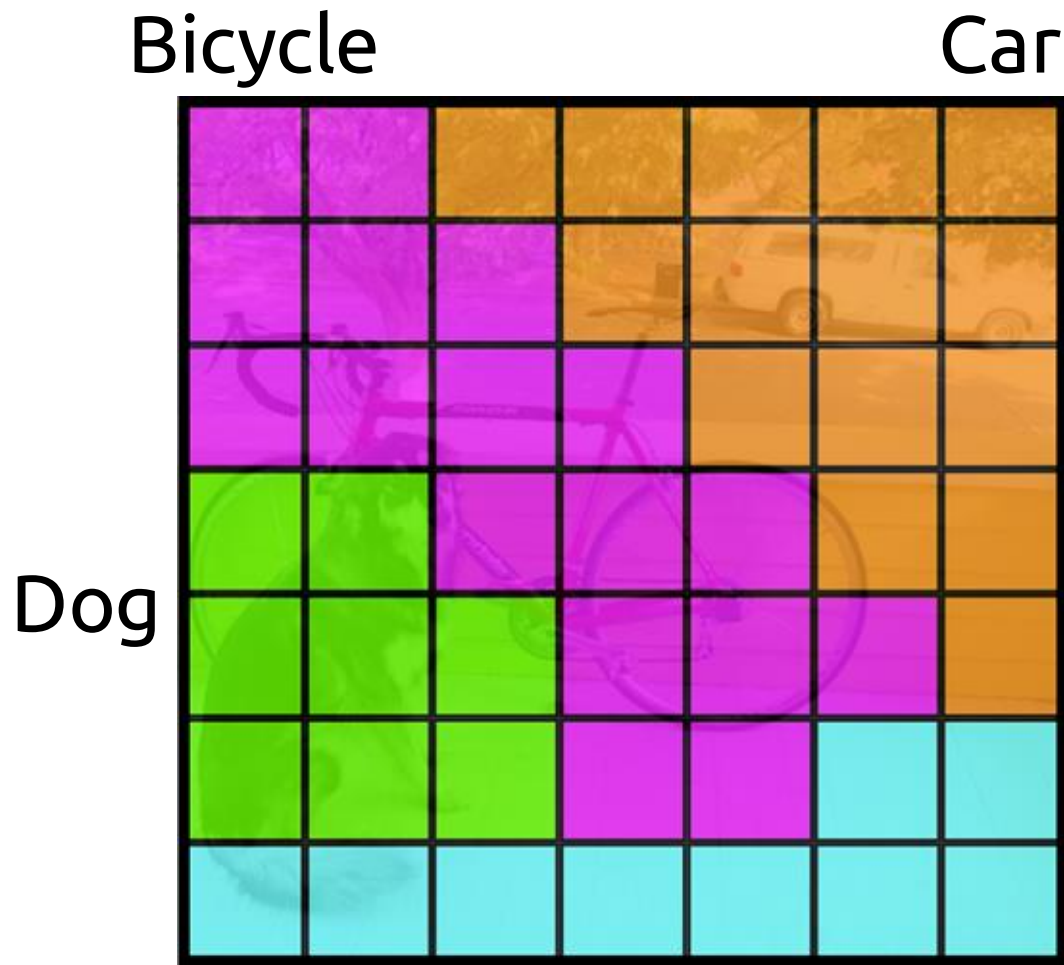


Divide image to 7 x 7 Grid

You Only Look Once



Bounding Boxes + Confidence

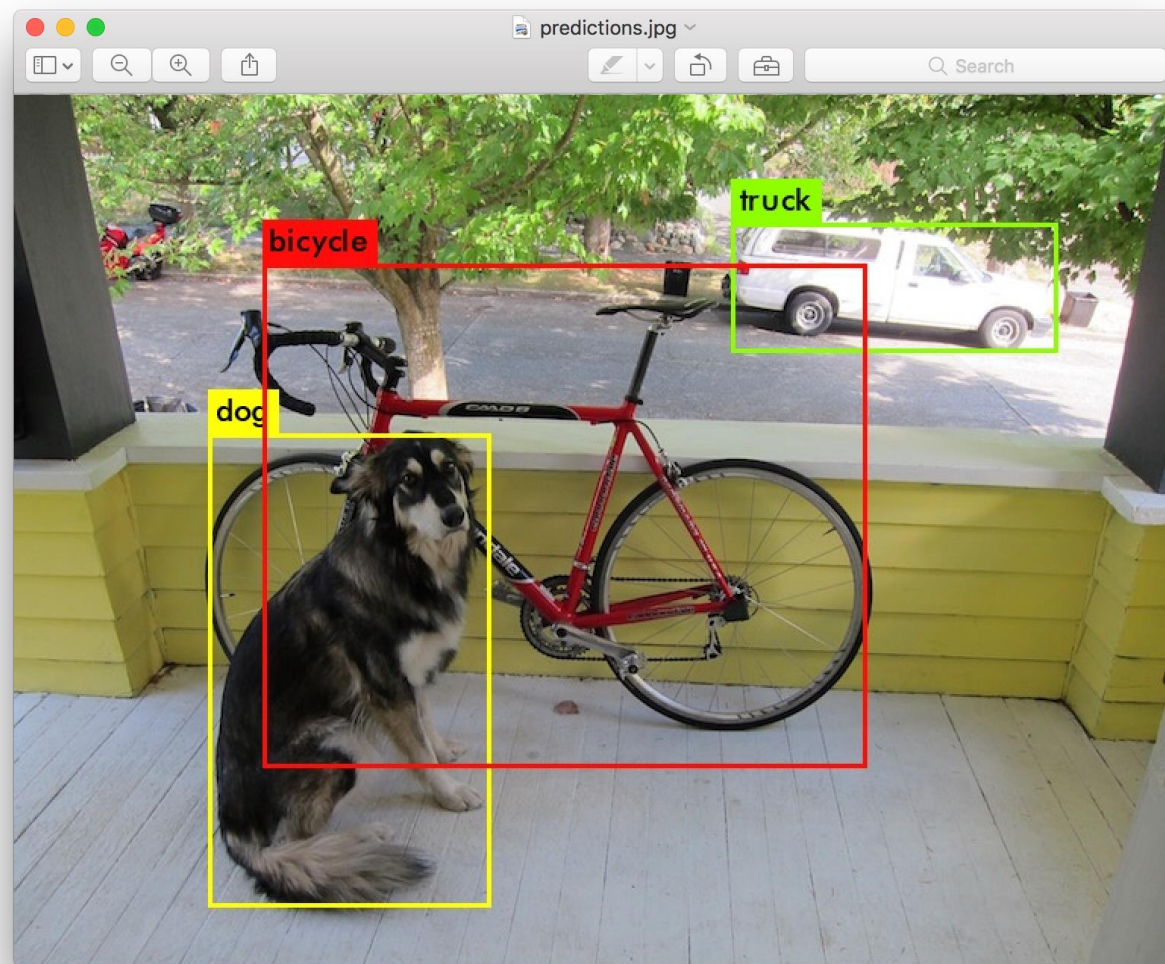


You Only Look Once



Bounding Boxes + Class Predictions

You Only Look Once



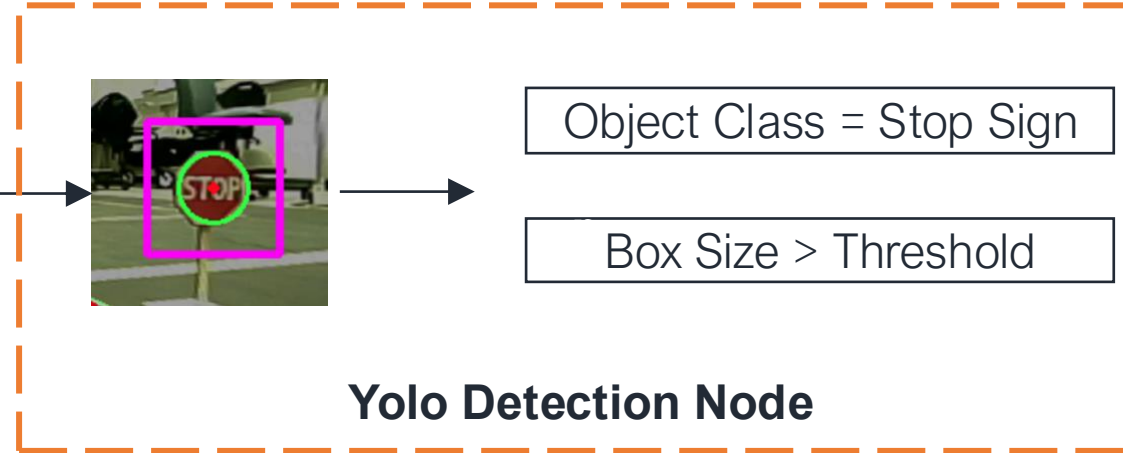
After Filtering and Non-Maximal Suppression

Today's Lab: Stop Sign Detection



Camera input

YOLO Object Detector



Driver

"Stop"

YOLO Detector ROS2 Node

- Don't forget to run Zed ROS2 Camera Node!
- Download this file:

`wget https://mankiyoon.github.io/courses/csc495/sp26/yolo\_detection\_node.py`

Task 1: View YOLO Results

TODO: Publish annotated image (HINT:
Refer Project 1)

TODO: Publish object count

TODO: View results using RVIZ2

```
# Callback
def image_callback(self, msg: Image):
    # Convert ROS Image to OpenCV BGR
    frame = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')

    # Run inference
    results = self.model(frame)
    annotated = results[0].plot()           # draws boxes on the frame
    num_objects = len(results[0].boxes)

    # TODO: Publish annotated image
    #####
    ## Your code here
    #####

    # TODO: Publish object count
    #####
    ## Your code here
    #####
```

Task 1: View YOLO Results

```
0: 384x640 1 person, 1 stop sign, 262.9ms
Speed: 3.6ms preprocess, 262.9ms inference, 1.9ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 1 stop sign, 246.8ms
Speed: 4.2ms preprocess, 246.8ms inference, 1.4ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 1 stop sign, 1 carrot, 242.3ms
Speed: 3.6ms preprocess, 242.3ms inference, 1.5ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 1 stop sign, 257.6ms
Speed: 3.4ms preprocess, 257.6ms inference, 1.3ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 1 stop sign, 264.8ms
Speed: 3.6ms preprocess, 264.8ms inference, 1.4ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 1 stop sign, 278.4ms
Speed: 3.6ms preprocess, 278.4ms inference, 1.4ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 1 stop sign, 278.9ms
Speed: 3.5ms preprocess, 278.9ms inference, 1.6ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 1 stop sign, 240.0ms
Speed: 3.8ms preprocess, 240.0ms inference, 1.4ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 1 stop sign, 253.0ms
Speed: 3.6ms preprocess, 253.0ms inference, 1.4ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 1 stop sign, 246.4ms
Speed: 3.5ms preprocess, 246.4ms inference, 1.3ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 1 stop sign, 259.4ms
Speed: 3.6ms preprocess, 259.4ms inference, 1.3ms postprocess per image at shape (1, 3, 384, 640)

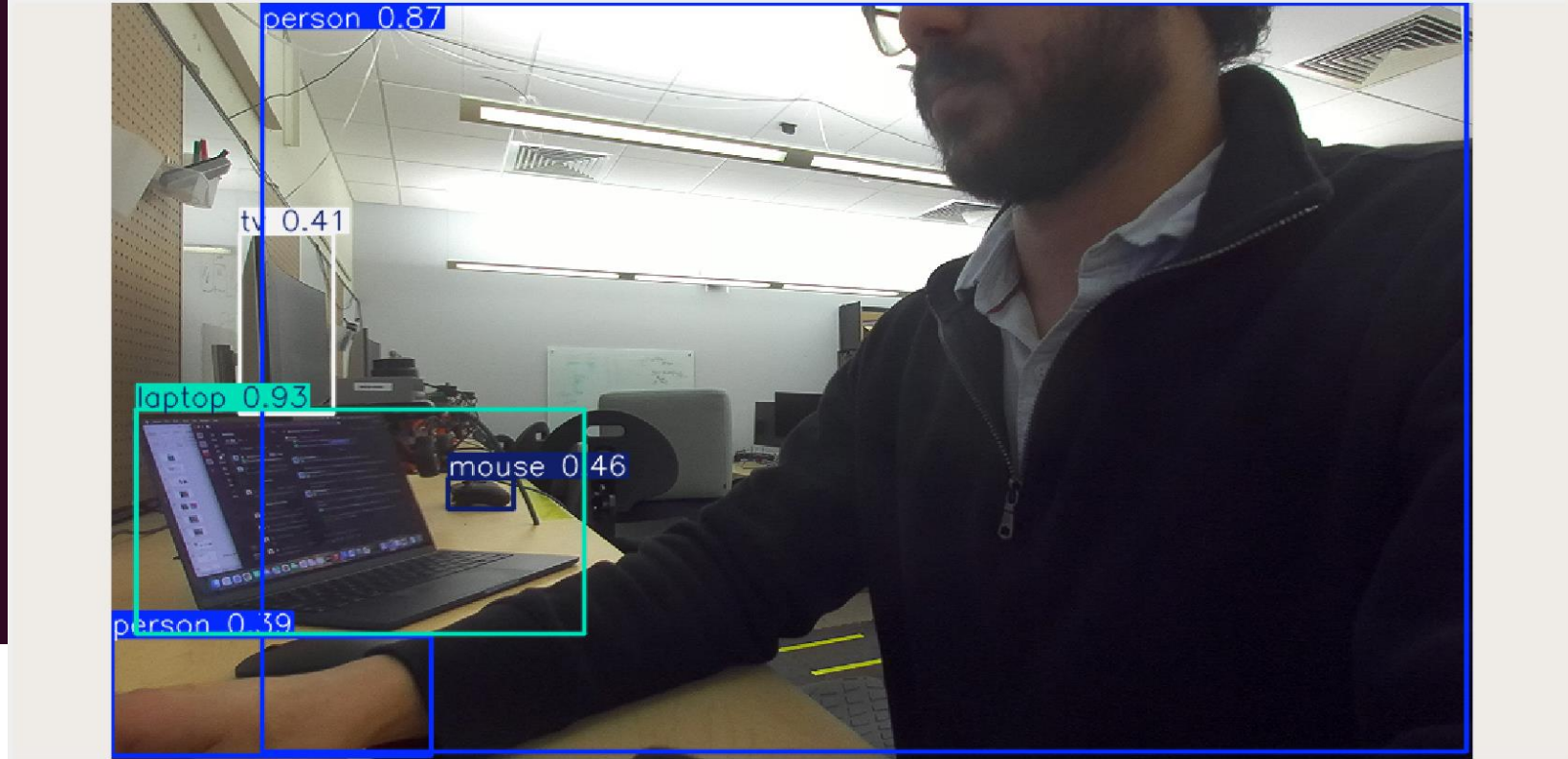
0: 384x640 1 person, 1 stop sign, 278.0ms
Speed: 3.5ms preprocess, 278.0ms inference, 1.7ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 1 stop sign, 261.6ms
Speed: 3.9ms preprocess, 261.6ms inference, 1.3ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 1 stop sign, 256.5ms
Speed: 3.7ms preprocess, 256.5ms inference, 1.7ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 1 stop sign, 246.9ms
Speed: 3.7ms preprocess, 246.9ms inference, 1.4ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 1 stop sign, 241.0ms
Speed: 3.5ms preprocess, 241.0ms inference, 1.3ms postprocess per image at shape (1, 3, 384, 640)
```



Task 2: Identify Stop Sign

TODO: Detect “Stop Sign” and its position and size (width, height)

HINT:

results[0].boxes.cls → class_ids

results[0].boxes.xyxy → top-left-x, top-left-y, bottom-right-x, bottom-right-y

results[0].boxes.names → {class_id: name} dict

```
# Run inference
results = self.model(frame)
annotated = results[0].plot() # draws boxes on the frame
num_objects = len(results[0].boxes)

# TODO: Publish annotated image
#####
## Your code here
#####

# TODO: Publish object count
#####
## Your code here
#####

# TODO: Get "Stop Sign" Size
# if area >= 300 * 300
# STOP
#####
## Your code here
#####
```


Task 2: Identify Stop Sign



(bottom-right-x, bottom-right-y)

Task 3: Block Throttle

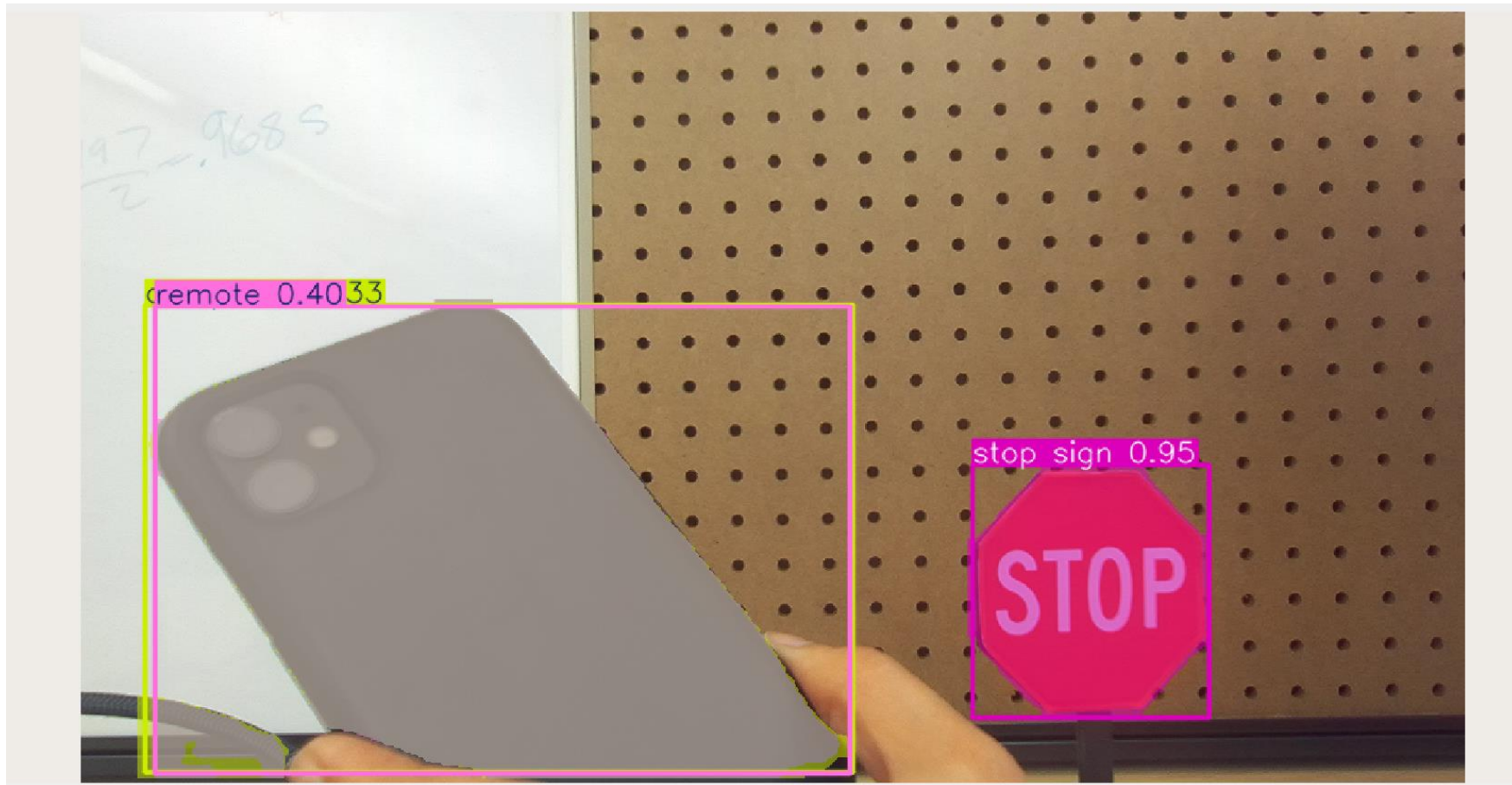
- If there is a “Stop Sign” and Size of “Stop Sign” $> 300 * 300$
 - Throttle = 0 in driver.py

```
self.sub = self.create_subscription(  
    Image, '/zed/zed_node/rgb/color/rect/image', self.image_callback, 10)  
  
self.pub_image = self.create_publisher(Image, '/detections/image', 10)  
self.pub_count = self.create_publisher(Int32, '/detections/count', 10)  
  
# TODO: Create topic to send "STOP" Signal  
#####  
## Your code here  
#####  
  
self.get_logger().info('YOLO detection node ready')
```

TODO: Show the TA or Professor!

(Optional) Task 4: YOLO Image Segmentation

- Change model to 'yolov8n-seg.pt'



(Optional) Task 4: YOLO Image Segmentation

`results[0].masks.xy → class_ids`

This prints the detection masks.
What can you do with this?



Explore More Resources!

×

ultralytics

Introducing the ultimate end-to-end vision AI platform

→

ultralytics

Ultralytics YOLO Docs

⚙️

🔖

🔍 Search

🔑

ultralytics/ultralytics

👁 v8.4.32

⭐ 55.2k

🗨 10.6k

Home

Modes

Tasks

Models

Compare

Datasets

Solutions

Guides

Integrations

Platform

Reference

Help

YOLOv3

YOLOv4

YOLOv5

YOLOv6

YOLOv7

YOLOv8

YOLOv9

YOLOv10

YOLO11

YOLO12

YOLO26 🚀

SAM (Segment Anything Model)

SAM 2 (Segment Anything Model 2)

SAM 3 (Segment Anything Model 3)

MobileSAM (Mobile Segment Anything Model)

Models Supported by Ultralytics

Welcome to Ultralytics' model documentation! We offer support for a wide range of models, each tailored to specific tasks like [object detection](#), [instance segmentation](#), [image classification](#), [pose estimation](#), and [multi-object tracking](#). If you're interested in contributing your model architecture to Ultralytics, check out our [Contributing Guide](#).

Model	Latency (ms/img)	COCO mAP50-95
YOLO26	2	42.5
YOLO26	4	52.5
YOLO26	6	55.0
YOLO26	12	57.5
YOLO11	2	40.0
YOLO11	4	50.0
YOLO11	6	52.5
YOLO11	12	55.0
YOLOv10	2	37.5
YOLOv10	4	47.5
YOLOv10	6	50.0
YOLOv10	12	52.5
YOLOv9	2	37.5
YOLOv9	4	45.0
YOLOv9	6	47.5
YOLOv9	12	50.0
YOLOv8	2	37.5
YOLOv8	4	45.0
YOLOv8	6	47.5
YOLOv8	12	50.0
YOLOv7	2	37.5
YOLOv7	4	45.0
YOLOv7	6	47.5
YOLOv7	12	50.0
YOLOv6-3.0	2	37.5
YOLOv6-3.0	4	45.0
YOLOv6-3.0	6	47.5
YOLOv6-3.0	12	50.0
YOLOv5	2	37.5
YOLOv5	4	45.0
YOLOv5	6	47.5
YOLOv5	12	50.0
PP-YOLOE+	2	37.5
PP-YOLOE+	4	45.0
PP-YOLOE+	6	47.5
PP-YOLOE+	12	50.0
DAMO-YOLO	2	37.5
DAMO-YOLO	4	45.0
DAMO-YOLO	6	47.5
DAMO-YOLO	12	50.0
RTMDet	2	37.5
RTMDet	4	45.0
RTMDet	6	47.5
RTMDet	12	50.0

On this page

Featured Models

Getting Started: Usage Examples

Contributing New Models

FAQ

What is the latest Ultralytics YOLO model?

How can I train a YOLO model on custom data?

Which YOLO versions are supported by Ultralytics?

Why should I use Ultralytics Platform for machine learning projects?

What types of tasks can Ultralytics YOLO models perform?

Ask AI

<https://docs.ultralytics.com/models/>

Explore More Resources!

[README](#) [MIT license](#)



HELLO AI WORLD NVIDIA JETSON

[Deploying Deep Learning](#)

Welcome to our instructional guide for inference and realtime vision [DNN library](#) for [NVIDIA Jetson](#) devices. This project uses [TensorRT](#) to run optimized networks on GPUs from C++ or Python, and PyTorch for training models.

Supported DNN vision primitives include [imageNet](#) for image classification, [detectNet](#) for object detection, [segNet](#) for semantic segmentation, [poseNet](#) for pose estimation, and [actionNet](#) for action recognition. Examples are provided for streaming from live camera feeds, making webapps with WebRTC, and support for ROS/ROS2.



97.07% polar bear

Image Classification



Object Detection



Semantic Segmentation



Pose Estimation

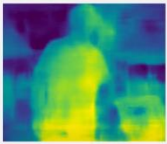


95.82% windsurfing

Action Recognition



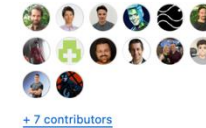
Background Removal



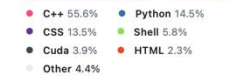
Mono Depth

Follow the [Hello AI World](#) tutorial for running inference and transfer learning onboard your Jetson, including collecting your own datasets, training your own models with PyTorch, and deploying them with TensorRT.

Table of Contents



Languages



<https://github.com/dusty-nv/jetson-inference>